

Parameterized Types and MSVC Complex Portability

Rohit Goswami · 2026-08-16 · python · fortran · numpy · f2py · ramblings

Precision is not a number. It is a contract between the programmer and the hardware. Fortran encodes this contract in the type system. C and Python do not. Bridging both gaps at once is what this post covers.

Background

Previous posts covered the three-layer architecture for wrapping non-`bind(c)` Fortran types. That pipeline assumed fixed types. So what happens when the Fortran type itself carries parameters?

The code corresponds to the [post/f2py-parameterized-types](#) tag.

Fortran 2003 introduced parameterized derived types (PDTs, F2018 7.5.3). A PDT is generic over integer type parameters that customize components at compile time (KIND parameters) or runtime (LEN parameters).

```
type :: RealVec(k)
  integer, kind :: k = kind(0.0d0)
  real(k) :: x, y, z
  integer :: length
end type RealVec
```

`RealVec` parameterizes the precision of its real members. The default `k = kind(0.0d0)` gives double precision, so `type(RealVec)` behaves like `type(RealVec(8))`. Set `k` to `4` and the coordinates become single precision.

I hit four separate problems while making PDTs compile through `f2py`. Each one needed its own fix. A fifth fix, on the MSVC side of complex number portability, landed in the same patch set because the generated code had to stay portable across GCC, Clang, and MSVC.

Scope

We cover:

1. How crackfortran parses (and previously failed to parse) PDT type specifiers
2. Kind expression resolution at two levels of the pipeline
3. Filtering type parameters from generated Python APIs
4. Compiler compatibility detection (flang-new does not support PDTs)
5. Replacing C11 `_Complex` with NumPy portable complex types

Manual wrappers: the multi-kind problem

Before looking at `f2py`'s pipeline, consider what we would write by hand. A `RealVec(8)` has `real(8)` members (C `double`). A `RealVec(4)` has `real(4)` members (C `float`). These are different C types, so we need separate wrapper sets for each KIND value:

► Multi-kind manual wrappers (click to expand)

Testing from C:

► C test (click to expand)